

RAG (Retrieval-Augmented Generation) — Study Notes

Covers: RAG basics, Retrieval Pipeline, Data Ingestion, and Typesense

1. What is RAG?

RAG = Retrieve + Generate. It is a technique where an AI system first searches for relevant information from documents or databases, and then generates an answer using that retrieved information.

Without RAG vs With RAG

Without RAG: The AI answers purely from its training memory — the answer may be wrong, generic, or outdated.

With RAG: The AI searches relevant documents/PDFs, finds the matching information, reads it, and then gives a grounded, correct answer.

Real-Life Analogy

Think of an **open-book exam**. Without RAG is like answering purely from memory. With RAG is like checking the book first, then answering — more accurate and reliable.

RAG = Retrieve + Generate



Figure 1: High-level RAG flow

Why Use RAG?

- Gives more accurate, grounded answers
- Uses the latest / private information (not just training data)
- Reduces hallucination (wrong/made-up answers)
- Can read and use PDFs, documents, and databases

Interview One-Liner

RAG is an AI method where the system first **retrieves** related information from a knowledge base, then **generates** an answer using that information.

2. Retrieval Pipeline

The retrieval pipeline is the **'search part'** of RAG — the process of finding the most relevant chunks of data from a knowledge base based on the user's query, before the LLM generates its answer.

Retrieval Pipeline (Step-by-Step)

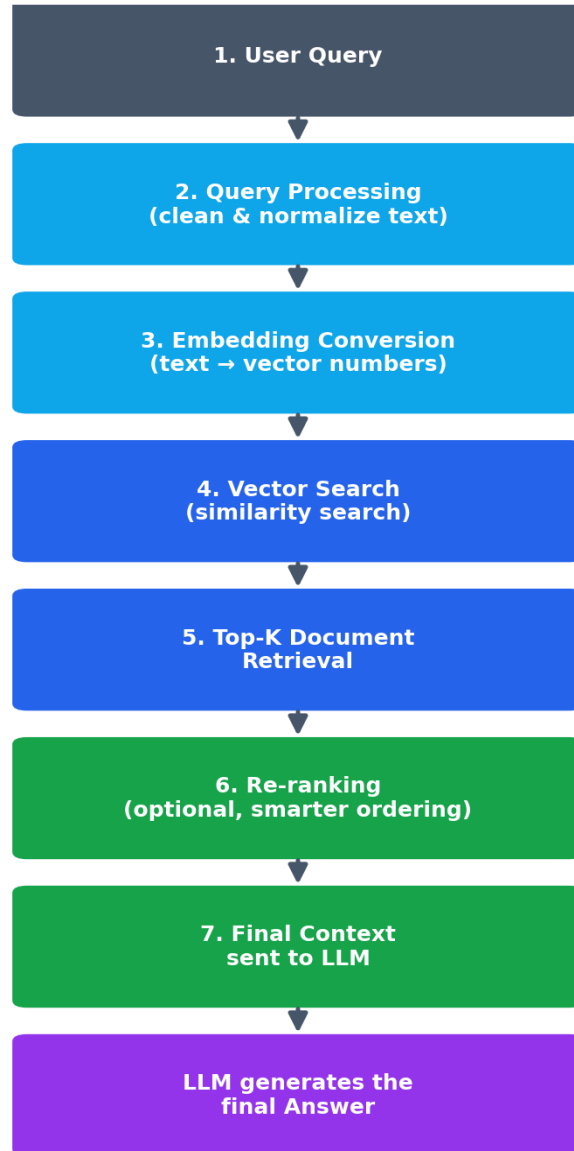


Figure 2: Step-by-step retrieval pipeline

Step Details

Step	What Happens
1. User Query	User asks a question, e.g. "What are Java exceptions?"
2. Query Processing	Clean the text — remove extra words, normalize formatting
3. Embedding Conversion	Convert text into a numeric vector representing its meaning

4. Vector Search	Compare the query vector with stored document vectors by similarity (meaning-based, not just keywords)
5. Top-K Retrieval	Select the top few most relevant chunks (e.g. top 3 documents)
6. Re-ranking (optional)	Use a smarter model to re-order results and pick the best ones
7. Send to LLM	Final selected context is passed to the LLM, which generates the answer

Analogy: The Librarian

You ask a question → the librarian (retrieval system) searches the books → picks the best pages → hands them to you (LLM) → you write the final answer using those pages.

Interview One-Liner

The retrieval pipeline converts a user query into a vector, searches a knowledge base for semantically similar information, and returns the most relevant documents for the LLM to generate an answer.

3. Data Ingestion Process

Data ingestion is the process of **collecting raw data from multiple sources** (PDFs, websites, APIs, databases, Excel files), cleaning and transforming it, and loading it into a storage system (database, data lake, or vector database) for further use.

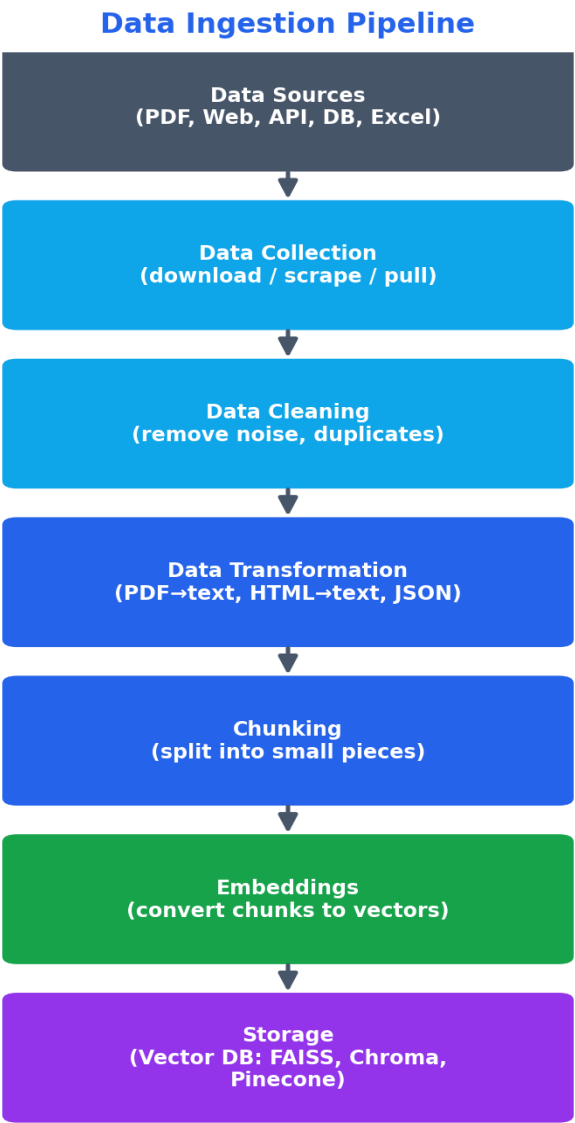


Figure 3: Data ingestion pipeline

Step Details

Step	Description
Data Sources	PDFs, websites, APIs, Excel files, databases
Data Collection	Download PDFs, scrape websites, pull DB records
Data Cleaning	Remove extra spaces, duplicates, errors, irrelevant content

Data Transformation	Convert formats: PDF → text, HTML → plain text, JSON → structured data
Chunking	Split large documents into smaller pieces (Chunk 1, Chunk 2, ...) so the AI works with manageable pa
Storage	Save into SQL/NoSQL database, data lake, or vector database (FAISS, Chroma, Pinecone)

Ingestion for RAG Specifically

In RAG systems, ingestion follows this flow: **Documents** → **Text** → **Chunks** → **Embeddings** → **Vector Database**. This prepares the data so the retrieval pipeline can later search it efficiently.

Types of Ingestion

- Batch Ingestion: data loaded at intervals (e.g. hourly/daily reports)
- Real-Time Ingestion: data arrives instantly (e.g. live chat messages, stock prices)

Interview One-Liner

Data ingestion is the process of collecting raw data from multiple sources, transforming it into a usable format, and storing it in a system for further processing such as analytics or AI applications.

4. Typesense

Typesense is a **fast, open-source search engine** used to build instant, Google-like search experiences inside applications — with typo tolerance, filters, and real-time results.

Typesense Search Flow



Figure 4: Typesense search flow

Key Features

- Fast Search: results return in milliseconds even on large datasets
- Typo Tolerance: e.g. "samsng" still matches "Samsung"
- Filtering / Faceted Search: filter by price, category, rating, etc.
- Relevance Ranking: most relevant results appear first
- Real-Time Indexing: newly added data is searchable immediately

Typesense vs Database Search

Feature	Database Search	Typesense
Speed	Slow	Very fast
Typo handling	No	Yes
Ranking	Basic	Smart ranking
Filters	Limited	Advanced
User experience	Poor	Google-like search

Real-World Use Cases

- E-commerce search (Amazon-like product search)
- Job portals
- Food delivery apps
- Blog / article search
- SaaS dashboards

Interview One-Liner

Typesense is a fast, open-source search engine that provides typo-tolerant, real-time, and highly relevant search results for applications.